



AES and SM4

Step-by-Step Encryption, Differential Fault Analysis, and Key Recovery

Educational Technical Notes for PFE 5A

Student: Guilherme Specht

Supervisor: Prof. Philippe Maurine

Institution: Polytech Montpellier

Industrial context: STMicroelectronics

Date: 20 October 2026

Contents

1 Purpose and Notation **3**

2 AES-128: Encryption from the Beginning **4**
2.1 Block, key, and round structure . . . 4
2.2 Initial AddRoundKey . . . 4
2.3 SubBytes . . . 4
2.4 ShiftRows . . . 5
2.5 MixColumns . . . 5
2.6 Round keys . . . 5

3 Piret–Quisquater Differential Fault Analysis of AES **6**
3.1 Where the fault is injected . . . 6
3.2 Propagation through MixColumns . . . 6
3.3 Which ciphertext bytes are affected? . . . 6
3.4 Constructing candidates for the last AES round key . . . 7
3.5 Intersecting observations . . . 7
3.6 Reversing AES-128 key expansion . . . 7

4 SM4: Complete Structure and First Round **8**
4.1 Official test vector . . . 8
4.2 SM4 round function . . . 8
4.3 SM4 key expansion . . . 8
4.4 Detailed computation of encryption round 0 . . . 9

5 Differential Fault Analysis of SM4: A Worked Last-Round Model **10**
5.1 Attack model and limitations . . . 10
5.2 Known ciphertext and hidden internal values . . . 10
5.3 Step 1: Inject a known XOR fault . . . 10
5.4 Step 2: Observe the S-box difference . . . 10
5.5 Step 3: Propagate the difference through L . . . 11
5.6 Step 4: Undo the linear layer from observed data . . . 11
5.7 Step 5: Convert S-box input candidates to key candidates . . . 11
5.8 Step 6: Use independent observations . . . 11
5.9 Step 7: Repeat for the other three bytes . . . 12
5.10 What happens after recovering the last subkey? . . . 12

6 SM4: Recovering the Master Key from Four Round Keys **13**
6.1 Required subkeys . . . 13
6.2 Reverse the recurrence . . . 13
6.3 Undo the initial constants . . . 13

7 AES versus SM4: Key Lessons **14**

References and Scope **15**

Chapter 1

Purpose and Notation

This document reorganizes the supplied handwritten AES/SM4 notes into a progressive English explanation. It distinguishes established algorithmic facts, pedagogical fault simulations, and the additional work needed for a physical attack.

- $\oplus$: bitwise XOR; $\lll n$: 32-bit circular left rotation by n bits.
- S: an 8-bit substitution box (AES and SM4 use different S-boxes).
- C: correct ciphertext; F: faulty ciphertext; $\Delta = C \oplus F$.
- All hexadecimal words are written most-significant byte first. SM4 words are 32 bits.

Note. A known plaintext/key test vector is used to *verify* examples. In an actual key-recovery experiment, the attacker does not know the secret key.

Chapter 2

AES-128: Encryption from the Beginning

2.1 Block, key, and round structure

AES is a 128-bit block cipher. AES-128, AES-192, and AES-256 use 10, 12, and 14 rounds respectively. We focus on AES-128.

Plaintext	`00112233445566778899AABBCCDDEEFF`
Key K_0	`000102030405060708090A0B0C0D0E0F`
Ciphertext	`69C4E0D86A7B0430D8CDB78070B4C55A`

AES stores bytes in a 4×4 *column-major* state:

$$\begin{pmatrix} 00 & 44 & 88 & CC \\ 11 & 55 & 99 & DD \\ 22 & 66 & AA & EE \\ 33 & 77 & BB & FF \end{pmatrix}.$$

First apply AddRoundKey with K_0. Rounds 1–9 apply SubBytes, ShiftRows, MixColumns, AddRoundKey. Round 10 omits MixColumns.

2.2 Initial AddRoundKey

The state is XORed byte by byte with the first round key. For example, $\texttt{11} \oplus \texttt{01} = \texttt{10}$. The result is

$$\begin{pmatrix} 00 & 40 & 80 & C0 \\ 10 & 50 & 90 & D0 \\ 20 & 60 & A0 & E0 \\ 30 & 70 & B0 & F0 \end{pmatrix}.$$

2.3 SubBytes

Each byte is independently replaced through the AES S-box. Examples: $S(\texttt{00}) = \texttt{63}$, $S(\texttt{10}) = \texttt{CA}$, $S(\texttt{20}) = \texttt{B7}$, $S(\texttt{30}) = \texttt{04}$. The resulting state is

$$\begin{pmatrix} 63 & 09 & CD & BA \\ CA & 53 & 60 & 70 \\ B7 & D0 & E0 & E1 \\ 04 & 51 & E7 & 8C \end{pmatrix}.$$

The S-box is nonlinear: in general, $S(a \oplus b) \neq S(a) \oplus S(b)$.

2.4 ShiftRows

Rows 0, 1, 2, 3 rotate left by 0, 1, 2, 3 bytes respectively:

$$\begin{pmatrix} 63 & 09 & CD & BA \\ 53 & 60 & 70 & CA \\ E0 & E1 & B7 & D0 \\ 8C & 04 & 51 & E7 \end{pmatrix}.$$

2.5 MixColumns

Every column is multiplied over $GF(2^8)$ by

$$M = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

For the first input column $(\texttt{63}, \texttt{53}, \texttt{E0}, \texttt{8C})^T$,

$$b_0 = (02 \cdot 63) \oplus (03 \cdot 53) \oplus E0 \oplus 8C.$$

Multiplication is in the AES finite field, not ordinary integer arithmetic. For a separate illustrative column $(\texttt{DB}, \texttt{13}, \texttt{53}, \texttt{45})^T$, MixColumns yields $(\texttt{8E}, \texttt{4D}, \texttt{A1}, \texttt{BC})^T$. The first column of the official AES round-1 state becomes $(\texttt{5F}, \texttt{72}, \texttt{64}, \texttt{15})^T$.

2.6 Round keys

AES-128 expands the original 16-byte key into eleven 16-byte round keys $K_0, \ldots, K_{10}$. For this vector:

$$K_1 = \texttt{D6AA74FDD2AF72FADAA678F1D6AB76FE},$$

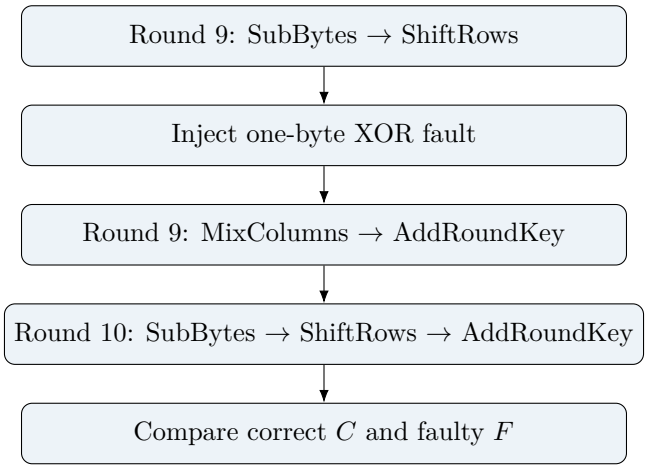
$$K_{10} = \texttt{13111D7FE3944A17F307A78B4D2B30C5}.$$

Chapter 3

Piret–Quisquater Differential Fault Analysis of AES

3.1 Where the fault is injected

In the classic model used in the notes, one nonzero byte is corrupted at the *input of MixColumns in round 9*, after SubBytes and ShiftRows. Both correct and faulty executions then proceed through round 9 and the final round.



3.2 Propagation through MixColumns

If the first byte of a column changes by e, the column difference after MixColumns is

$$\Delta_{MC} = (02e,\ e,\ e,\ 03e)^T.$$

For $e = \texttt{01}$, this is $(\texttt{02}, \texttt{01}, \texttt{01}, \texttt{03})^T$. AddRoundKey does not alter XOR differences because both executions use the same round key. The final SubBytes transforms these differences nonlinearly; ShiftRows moves them into separate ciphertext positions.

3.3 Which ciphertext bytes are affected?

For the four round-9 columns, the affected ciphertext positions (in corresponding row order) are:

Faulted column	Ciphertext byte positions
0	0, 13, 10, 7
1	4, 1, 14, 11
2	8, 5, 2, 15
3	12, 9, 6, 3

3.4 Constructing candidates for the last AES round key

For an affected ciphertext position j, and a guessed last-round key byte k, compute

$$d_j(k) = S^{-1}(C_j \oplus k) \oplus S^{-1}(F_j \oplus k).$$

Each byte has 256 candidate values. Four unknown key bytes have $256^4 = 2^{32}$ possible combinations before filtering. Construct a set $\mathcal{D}$ containing the MixColumns output differences for a single nonzero faulty byte in any of the four row positions:

$$\mathcal{D} = \{M(e\mathbf{e}_r) : r \in \{0, 1, 2, 3\},\ e \in \{1, \ldots, 255\}\}.$$

This produces $4 \cdot 255 = 1020$ four-byte difference patterns (not 1020 individual bytes). For each proposed key quartet, compute the four $d_j(k)$ values in their correct *pre-ShiftRows row order* and keep the quartet only if the resulting vector lies in $\mathcal{D}$.

3.5 Intersecting observations

A single correct/faulty pair typically leaves multiple candidate key quartets. Repeat with independent faults in the same column and intersect the candidate sets:

$$\mathcal{K}_{\text{surviving}} = \mathcal{K}_1 \cap \mathcal{K}_2 \cap \cdots .$$

Repeat across all four columns to recover the full 16-byte K_{10}. The notes illustrate candidate filtering using a hypothetical quartet (`13`, `2B`, `A7`, `17`); such pedagogical candidate examples should not be confused with a complete independently verified AES fault trace.

3.6 Reversing AES-128 key expansion

Let $w_0, \ldots, w_{43}$ be 32-bit expanded-key words. For $i \geq 4$,

$$w_i = \begin{cases} w_{i-4} \oplus g(w_{i-1}), & i \equiv 0 \pmod 4, \\ w_{i-4} \oplus w_{i-1}, & \text{otherwise}, \end{cases}$$

where $g(w) = \text{SubWord}(\text{RotWord}(w)) \oplus \text{Rcon}$. In reverse, compute w_{i-4} by XORing w_i with the same known right-hand transformation. Begin with $w_{40}, \ldots, w_{43}$ from K_{10} and work backward to $w_0, \ldots, w_3$, yielding K_0. No inverse S-box is required for this reversal because the relevant inputs to g are already known.

Chapter 4

SM4: Complete Structure and First Round

4.1 Official test vector

Plaintext	`0123456789ABCDEFFEDCBA9876543210`
Master key	`0123456789ABCDEFFEDCBA9876543210`
Ciphertext	`681EDF34D206965E86B3E94F536E4246`

Split the plaintext into four 32-bit words:

$$X_0 = \texttt{01234567}, \quad X_1 = \texttt{89ABCDEF}, \quad X_2 = \texttt{FEDCBA98}, \quad X_3 = \texttt{76543210}.$$

4.2 SM4 round function

SM4 performs 32 rounds, producing one new 32-bit word per round:

$$\boxed{X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)}.$$

Write $T = L \circ \tau$, where τ substitutes the four bytes through the SM4 S-box and

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

The output reverses the final four words:

$$C = X_{35} \parallel X_{34} \parallel X_{33} \parallel X_{32}.$$

4.3 SM4 key expansion

The four initial key-schedule words are $K_i = MK_i \oplus FK_i$, with

$$FK_0 = \texttt{A3B1BAC6}, \quad FK_1 = \texttt{56AA3350},$$
$$FK_2 = \texttt{677D9197}, \quad FK_3 = \texttt{B27022DC}.$$

The key-schedule recurrence is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i), \quad rk_i = K_{i+4},$$

with $T' = L' \circ \tau$ and

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23).$$

Crucial distinction: encryption uses L, while key expansion uses L'. They have different rotation constants. For the first round, $rk_0 = \texttt{F12186F9}$.

4.4 Detailed computation of encryption round 0

Step 1: XOR the three newest words with the round key.

$$A = X_1 \oplus X_2 \oplus X_3 \oplus rk_0 = \texttt{F002C39E}.$$

Step 2: Apply the SM4 S-box to each byte.

$$\texttt{F0} \mapsto \texttt{18}, \quad \texttt{02} \mapsto \texttt{E9}, \quad \texttt{C3} \mapsto \texttt{92}, \quad \texttt{9E} \mapsto \texttt{B1}.$$

Thus $B = \tau(A) = \texttt{18E992B1}$.

Step 3: Apply the linear transformation.

Term	Hexadecimal value
B	18E992B1
$B \lll 2$	63A64AC4
$B \lll 10$	A64AC463
$B \lll 18$	4AC463A6
$B \lll 24$	B118E992
XOR of all five	26D99622

Step 4: XOR with the oldest word.

$$X_4 = X_0 \oplus L(B) = \texttt{01234567} \oplus \texttt{26D99622} = \boxed{\texttt{27FAD345}}.$$

The next four-word window is (X_1, X_2, X_3, X_4). The following words are $X_5 = \texttt{A18B4CB2}$, $X_6 = \texttt{11C1E22A}$, and $X_7 = \texttt{CC13E2EE}$. After 32 rounds and the output reversal, the official ciphertext is obtained.

Chapter 5

Differential Fault Analysis of SM4: A Worked Last-Round Model

5.1 Attack model and limitations

We use a *controlled, known XOR fault* in one byte of the input to the SM4 S-boxes in round 31. This is a pedagogical model, not a claim that a real clock/voltage glitch yields a known XOR value. In this model, all other round inputs remain unchanged. The last round is

$$X_{35} = X_{31} \oplus L(\tau(A)), \qquad A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}.$$

The attacker observes the correct ciphertext and a faulty ciphertext. The secret internal values below are supplied *only for checking the simulation*.

5.2 Known ciphertext and hidden internal values

$$X_{35} = \texttt{681EDF34}, \quad X_{34} = \texttt{D206965E},$$
$$X_{33} = \texttt{86B3E94F}, \quad X_{32} = \texttt{536E4246}.$$

For this official test vector, the simulation has

$$X_{31} = \texttt{7B938F4C}, \quad A = \texttt{96FF9D45}, \quad rk_{31} = \texttt{9124A012}.$$

The attacker must recover the key rather than assume it.

5.3 Step 1: Inject a known XOR fault

Alter the first byte of A by $e = \texttt{01}$:

$$A = \texttt{96FF9D45} \longrightarrow A' = \texttt{97FF9D45}.$$

5.4 Step 2: Observe the S-box difference

The two S-box outputs are

$$B = \tau(A) = \texttt{1A488C73}, \qquad B' = \tau(A') = \texttt{55488C73}.$$

Consequently,

$$\Delta B = B \oplus B' = \texttt{4F000000}.$$

5.5 Step 3: Propagate the difference through L

Because L is XOR-linear,

$$\Delta X_{35} = L(B) \oplus L(B') = L(\Delta B) = \texttt{734E3D3D}.$$

The correct last word is $X_{35} = \texttt{681EDF34}$, so the faulty last word is

$$X'_{35} = \texttt{681EDF34} \oplus \texttt{734E3D3D} = \texttt{1B50E209}.$$

Therefore the attacker sees

Correct ciphertext	681EDF34	D206965E	86B3E94F	536E4246
Faulty ciphertext	1B50E209	D206965E	86B3E94F	536E4246
Difference	734E3D3D	00000000	00000000	00000000

5.6 Step 4: Undo the linear layer from observed data

The attacker computes

$$\Delta B = L^{-1}(\Delta X_{35}) = L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}.$$

The first S-box byte therefore satisfies

$$S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}.$$

Enumerating all 256 possible input bytes gives the candidate set

$$\mathcal{A}_1 = \{\texttt{96}, \texttt{97}\}.$$

5.7 Step 5: Convert S-box input candidates to key candidates

The attacker knows the last three words from the ciphertext:

$$X_{32} \oplus X_{33} \oplus X_{34} = \texttt{07DB3D57}.$$

Since $rk_{31} = A \oplus \texttt{07DB3D57}$, the first byte of the key is

$$rk_{31}[0] = a \oplus \texttt{07}.$$

Thus $a = \texttt{96}$ yields $\texttt{91}$, while $a = \texttt{97}$ yields $\texttt{90}$. After one observation, the candidates are $\{\texttt{91}, \texttt{90}\}$.

5.8 Step 6: Use independent observations

Repeat with the *same plaintext and state*, but use a known XOR fault $e = \texttt{02}$. The new S-box difference is $d_2 = \texttt{81}$. Keep only input candidates satisfying both

$$\begin{cases} S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}, \\ S(a) \oplus S(a \oplus \texttt{02}) = \texttt{81}. \end{cases}$$

The intersection identifies $a = \texttt{96}$, hence $rk_{31}[0] = \texttt{91}$.

5.9 Step 7: Repeat for the other three bytes

The supplied worked example reports the following bytewise observations under the same controlled-fault assumption:

Byte	d for $e = 01$	d for $e = 02$	Recovered A	Recovered rk_{31}
0	4F	81	96	91
1	71	83	FF	24
2	79	6F	9D	A0
3	80	C9	45	12

This reconstructs

$$A = \texttt{96FF9D45}, \qquad rk_{31} = A \oplus \texttt{07DB3D57} = \boxed{\texttt{9124A012}}.$$

Note. This illustrates last-round subkey recovery *conditional on the fault model*. The listed observations should be validated against a software implementation before treating them as experimental results.

5.10 What happens after recovering the last subkey?

Knowing rk_{31}, reconstruct the previous state word:

$$X_{31} = X_{35} \oplus T(X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}).$$

For the example, $X_{31} = \texttt{7B938F4C}$. However, this does *not* automatically reveal rk_{30}. Additional fault observations and state constraints are required for rounds 30, 29, and 28. The present worked example does not demonstrate that stage of the attack.

Chapter 6

SM4: Recovering the Master Key from Four Round Keys

6.1 Required subkeys

If four consecutive final subkeys have been recovered, then

Key-schedule word	Round key
K_{32}	$rk_{28} =$ `428D3654`
K_{33}	$rk_{29} =$ `62293496`
K_{34}	$rk_{30} =$ `01CF72E5`
K_{35}	$rk_{31} =$ `9124A012`

These are reference subkeys from the official example; *only* rk_{31} was recovered in the preceding pedagogical DFA walkthrough.

6.2 Reverse the recurrence

The forward key schedule is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i).$$

XOR is self-inverse, so

$$\boxed{K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)}.$$

Start with $i = 31$, then decrease to $i = 0$. For example,

$$K_{31} = K_{35} \oplus T'(K_{32} \oplus K_{33} \oplus K_{34} \oplus CK_{31}),$$

where $CK_{31} =$ `646B7279`. This yields the previous expanded-key word $K_{31} =$ `F1780C81` in the official test vector. Continue to recover K_0, K_1, K_2, K_3.

6.3 Undo the initial constants

Finally,

$$MK_i = K_i \oplus FK_i \qquad (i = 0, 1, 2, 3),$$

and concatenate the four master-key words:

$$\boxed{MK = \texttt{0123456789ABCDEFFEDCBA9876543210}}.$$

Chapter 7

AES versus SM4: Key Lessons

Property	AES-128	SM4
Block size	128 bits	128 bits
Key size	128 bits	128 bits
Rounds	10	32
State	4×4 byte matrix	Four 32-bit words
Nonlinearity	AES S-box	SM4 S-box
Linear diffusion	MixColumns and ShiftRows	Rotations and XOR in L
Last-round target	K_{10}, 128 bits	rk_{31}, 32 bits
Master-key recovery	Reverse AES expansion	Reverse SM4 expansion from four consecutive round keys

Summary

For AES Piret–Quisquater, compare correct and faulty ciphertexts, invert the last S-box under candidate K_{10} bytes, match round-9 MixColumns differential patterns, intersect candidates, and reverse the AES key schedule.

For the illustrated SM4 model, compare correct and faulty ciphertexts, invert the linear map L, test S-box input candidates using the known fault differences, derive rk_{31}, and obtain further subkeys with additional appropriate experiments. Once four consecutive round keys are known, the SM4 key schedule can be reversed exactly.

References and Scope

1. Student-provided handwritten notes, *AES.pdf*, 18 pages, including AES fundamentals, Piret–Quisquater, SM4 fundamentals, and an SM4 DFA example.
2. NIST, *FIPS 197: Advanced Encryption Standard (AES)* (algorithm and official AES test vector).
3. ISO/IEC 18033-3, *Encryption algorithms — Part 3: Block ciphers* (SM4 specification); SM4 test vector as reproduced in the supplied notes.

Note. The SM4 DFA chapter uses a deliberately strong fault assumption: known XOR magnitude at a specified S-box input. Experimental fault injection must establish whether that assumption is justified. No full experimental recovery of all four SM4 final round keys is claimed here.